Language Fundamentals

- **Python is a general-purpose high-level programming language.**
- **Python was developed by Guido Van Rossam**
- **officially Python was made available to public in 1991**
- **Python is recommended as first programming language for beginners.**

Where we can use Python

1. For developing Web Applications
2. For developing Desktop Applications
3. For developing Database Applications
4. For developing Network Programming
5. For developing Games
6. For developing Data Analysis applications
7. For developing Gen AI / Machine Learning applications
8. For developing IOT applications

Features of Python:

- **Simple and easy to learn**
  ➔ Python is a simple programming language. When we read Python program, we can feel like reading English statements.

- **Open Source**
  ➔ We can use Python software without any licence, and it is freeware.

- **High Level Programming language**
  → Python is high level programming language and hence it is programmer friendly language.

- **Platform Independent**
  ➔ Once we write a Python program, it can run on any platform without rewriting once again.

- **Portability**
  ➔ Python programs are portable. i.e. we can migrate from one platform to another platform very easily. Python programs will provide same results on any platform.

- **Dynamically Typed**
  ➔ In Python we are not required to declare type for variables. Whenever we are assigning the value, based on value, type will be allocated automatically. Hence Python is considered as dynamically typed language.

- **Both Procedure Oriented and Object Oriented**
  ➔ Python language supports both Procedure oriented (like C, pascal etc) and object oriented (like C++, Java) features. Hence, we can get benefits of both like security and reusability etc.

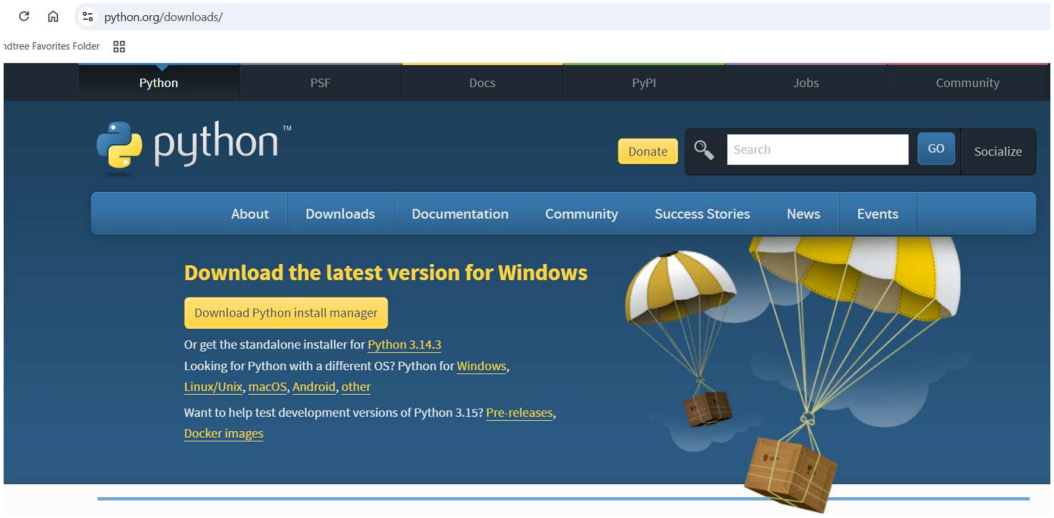
- **Interpreted**
  ➔ We are not required to compile Python programs explicitly. Internally Python interpreter will take care that compilation.

- **Extensible**
  ➔ We can use other language programs in Python

- **Embedded**
  - ➔ We can use Python programs in any other language programs

How to Download and Install and Set Environment Variables in Window Laptop

Step 1: Go to python official website and download Python [get standalone installer]



Step 2: Download it and install it. Remember the path.

Step 3: Once it will be installed, by default below path taken

C:\Users\<YourUserName>\AppData\Local\Programs\Python\PythonXX\ (where XX is the version, e.g., Python311).
The AppData folder is often hidden, so you may need to adjust your folder view settings to see it.

Step 4: Set Environment Variable

a. Find the installation path (e.g., C:\Users\YourName\AppData\Local\Programs\Python\Python311\).
b. Search for "environment variables" in the Start menu and select "Edit the system environment variables".
c. Click the "Environment Variables..." button.
d. Under "User variables" (or "System variables" if you have admin rights), find the Path variable and click "Edit".
e. Click "New" and paste the path to your Python installation folder.
f. Add another new entry for the Scripts folder within your Python installation (e.g., C:\Users\YourName\AppData\Local\Programs\Python\Python311\Scripts\).
g. Click "OK" on all windows to apply the changes.
h. Open a *new* Command Prompt or PowerShell window and type **python --version** to verify the path is set correctly.

```
C:\Users\10841188>python --version
Python 3.14.0

C:\Users\10841188>
```

Identifiers

- A Name in Python Program is called Identifier.
- It can be Class Name OR Function Name OR Module Name OR Variable Name.

Rules to define Identifiers in Python:

1. alphabet symbols (either lower case or upper case)
   digits (0 to 9)
   underscore symbol (_)

2. Identifier should not start with digit

3. Identifiers are case sensitive. Python language is case sensitive language.
   Ex: - abc = 10
   ABC = 10
   Print(abc) → 10
   Print (ABC) → 10

4. We cannot use reserved words as identifiers

5. There is no length limit for Python identifiers. But not recommended to use too lengthy identifiers.

6. If identifier starts with _ symbol, then it indicates that it is private

7. If identifier starts with __ (Two Under Score Symbols) indicating that strongly private identifier.

8. If the identifier starts and ends with two underscore symbols then the identifier is language defined special name, which is also known as magic methods.
   Ex: __add__, __init__

For Practice…Check valid identifiers?

a. 567abc
b. 123498
c. _wer
d. Abcf678_
e. Dfgki
f. Def
g. If
h. True
i. _abc_abc
j. Python2java
k. Wep458
l. Car_bike_345
m. 456_bike
n. __dty
o. I_am_&variable
p. Ty$kj98

q. Py78v%23&

Reserved Words

In Python some words are reserved to represent some meaning or functionality. Such types of words are called reserved words.

- All Reserved words in Python contain only alphabet symbols
- Except the following 3 reserved words, all contain only lower-case alphabet symbols.
  True
  False
  None

Data Types

Data Type represents the type of data present inside a variable.

Python contains the following inbuilt data types
1) Int
2) Float
3) Complex
4) Bool
5) Str
6) Bytes
7) Bytearray
8) Range
9) List
10) Tuple
11) Set
12) Frozenset
13) Dict
14) None

Python Inbuilt functions

1. type ()
   to check the type of variable
2. id ()
   to get address of object
3. print ()
   to print the value

int Data Type

- We can use int data type to represent whole numbers
- We can represent int values in the following ways
  1) Decimal form
  2) Binary form
  3) Octal form
  4) Hexa decimal form

  Examples:-

  num_binary = 0b1011 / 0B1011
  print(num_binary)
  Prefix(es) for Binary => 0b or 0B

```
num_binary = 0b1011
print(num_binary)
```

```
11

=== Code Execution Successful ===
```

```
num_binary = 0B1011
print(num_binary)
```

```
11

=== Code Execution Successful ===
```

num_octal = 0o27
print(num_octal)
Prefix(es) for Octal => 0o or 0O

```
num_octal = 0o27
print(num_octal)

num_octal = 0O27
print(num_octal)
```

```
23
23

=== Code Execution Successful ===
```

num_hex = 0xAF
print(num_hex)
Prefix(es) for Hexadecimal => 0x or 0X

```
num_hex = 0xAF
print(num_hex)

num_hex = 0XAF
print(num_hex)
```

```
175
175

=== Code Execution Successful ===
```

num_decimal = 126789
print(num_decimal)

```
num_decimal = 126789
print(num_decimal)
```

```
126789

=== Code Execution Successful ===
```

- Python offers the following built-in functions for base conversions
  - bin ()
  - oct ()
  - hex ()

Float Data Type

We can use float data type to represent floating point values (decimal values)

```
main.py
num_float = 126789.8765490
print(num_float)
print(id(num_float))
print(type(num_float))
```

```
Output
126789.876549
135539227330704
<class 'float'>

=== Code Execution Successful ===
```

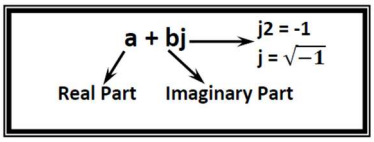
- binary, octal and hexa decimal forms are not supported for float.

```
main.py
num_float = 0B11.01
print(num_float)
print(id(num_float))
print(type(num_float))
```

```
Output
ERROR!
Traceback (most recent call last):
  File "<main.py>", line 1
    num_float = 0B11.01
                   ^^^
SyntaxError: invalid syntax
```

Complex Data Type

A complex number is of the form



'a' and 'b' contain Integers OR Floating-Point Values.

Ex:- 12+13j
12+8.9j
1.2+4.7j

```
main.py
num_complex = 12+13j
num_complex_1 = 12+8.9j
num_complex_2 = 1.2+4.7j
print(num_complex)
print(num_complex_1)
print(num_complex_2)
print(type(num_complex))
print(type(num_complex_1))
print(type(num_complex_2))
```

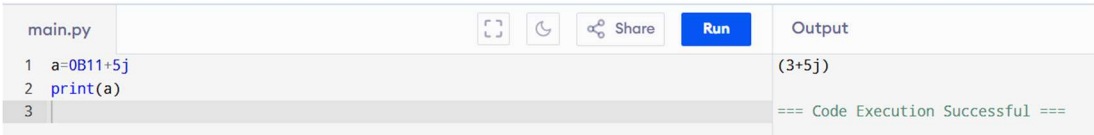
```
Output
(12+13j)
(12+8.9j)
(1.2+4.7j)
<class 'complex'>
<class 'complex'>
<class 'complex'>

=== Code Execution Successful ===
```

- In the real part if we use int value then we can specify that either by decimal, octal, binary or hexa decimal form.
- But imaginary part should be specified only by using decimal form.

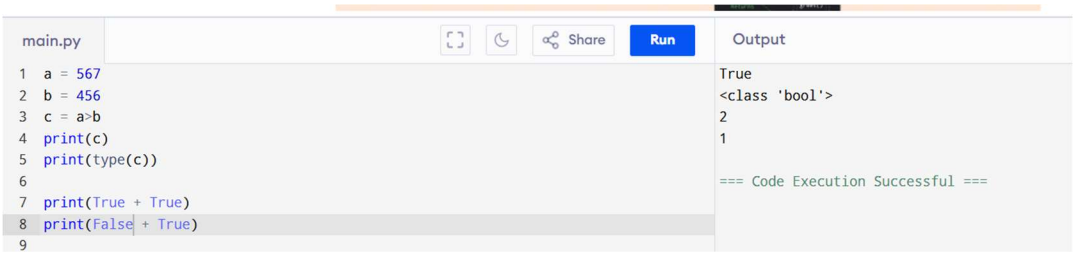
Example:

```
a=0B11+5j
print(a)
```



bool Data Type

- We can use this data type to represent Boolean values.
- The only allowed values for this data type are:
  True and False
- Internally Python represents True as 1 and False as 0



str Data Type

A String is a sequence of characters enclosed within single quotes or double quotes or triple quotes.

S = “Python”

S= “”” Python”””

S= ‘Python’

```python
s= "python"

s1 = 'python'

s2 = '''python'''

print(s)
print(s1)
print(s2)

print(type(s))
print(type(s1))
print(type(s2))
```

```
python
python
python
<class 'str'>
<class 'str'>
<class 'str'>

=== Code Execution Successful ===
```

In Python the following data types are considered as Fundamental Data types

- ✓ int
- ✓ float
- ✓ complex
- ✓ bool
- ✓ str

bytes Data Type

- bytes data type represents a group of byte numbers just like an array.
- The only allowed values for byte data type are 0 to 256. By mistake if we are trying to provide any other values then we will get value error.
- Once we create bytes data type value, we cannot change its values, otherwise we will get TypeError.

```python
a = [215, 66, 67]

 # Converts the list of integers into bytes
b = bytes(a)
print(b)
print(type(b))
```

```
b'\xd7BC'
<class 'bytes'>

=== Code Execution Successful ===
```

```python
a = [215, 66, 67]

 # Converts the list of integers into bytes
b = bytes(a)
b[1] = 126
print(b)
print(type(b))
```

```
ERROR!
Traceback (most recent call last):
  File "<main.py>", line 5, in <module>
TypeError: 'bytes' object does not support item assignment

=== Code Exited With Errors ===
```

bytearray Data Type:

bytearray is exactly same as bytes data type except that its elements can be modified.

main.py

```python
a = [215, 66, 67, 87, 90]

 # Converts the list of integers into bytes array
b = bytearray(a)
b[1] = 126
print(b)
print(type(b))

for i in b:
    print(i, end = ',')
```

Output

```
bytearray(b'\xd7~CWZ')
<class 'bytearray'>
215,126,67,87,90,
=== Code Execution Successful ===
```

List Data Type

In Python, a list is a built-in data structure that can hold an ordered collection of items.

- Insertion Order is preserved
- Heterogeneous Objects are allowed
- Duplicates are allowed
- Growable in nature
- Values should be enclosed within square brackets.

L1 = [“Python”, 10, 23, 789, 567.9078, True, False, 0, 1,1,1,1,1]

main.py

```python
L1 = ["Python", 10, 23, 789, 567.9078, True, False, 0, 1,1,1,1,1]
print(L1)
print(id(L1))
print(type(L1))
```

Output

```
['Python', 10, 23, 789, 567.9078, True, False, 0, 1, 1, 1, 1, 1]
140388198353344
<class 'list'>

=== Code Execution Successful ===
```

Tuple Data Type:

- Tuple data type is exactly same as list data type except that it is immutable, we cannot change values.
- Tuple elements can be represented within parenthesis.

main.py

```python
T1 = ("Python", 10, 23, 789, 567.9078, True, False, 0, 1,1,1,1,1)
print(T1)
print(id(T1))
print(type(T1))
```

Output

```
('Python', 10, 23, 789, 567.9078, True, False, 0, 1, 1, 1, 1, 1)
134245667479632
<class 'tuple'>

=== Code Execution Successful ===
```

Range Data Type

- range Data Type represents a sequence of numbers.
- Immutable data type

➢ range (10)
generate numbers from 0 to 9

```
x = range(10)

for i in x:
    print(i, end = ",")
```

```
0,1,2,3,4,5,6,7,8,9,
=== Code Execution Successful ===
```

➢ range (10, 20)
generate numbers from 10 to 19

main.py | Share | Run | Output

```
x = range(10, 20)

for i in x:
    print(i, end = ",")
```

```
10,11,12,13,14,15,16,17,18,19,
=== Code Execution Successful ===
```

➢ r = range (10,20,2)
It will generate number skipping next one.

main.py | Share | Run | Output

```
x = range(10, 20, 2)

for i in x:
    print(i, end = ",")
```

```
10,12,14,16,18,
=== Code Execution Successful ===
```

set Data Type

Python set is an unordered collection of multiple items having different datatypes. In Python, sets are mutable, unindexed and do not contain duplicates.

- Duplicates are not allowed
- Insertion order is not preserved
- Heterogeneous objects are allowed
- Index concept is not applicable
- Growable in nature

```
set1 = {1, 2, 3, 4}
print(set1)
print(type(set1))
print(id(set1))
```

main.py | Share | Run | Output

```
set1 = {1, 2, 3, 4}
print(set1)
print(type(set1))
print(id(set1))
```

```
{1, 2, 3, 4}
<class 'set'>
136129772851872

=== Code Execution Successful ===
```

frozenset Data Type

- It is exactly same as set except that it is immutable.
- Hence, we cannot use add or remove functions.

main.py

```
s={10,20,30,40}
fs=frozenset(s)
print(type(fs))
print(fs)
```

Output

```
<class 'frozenset'>
frozenset({40, 10, 20, 30})

=== Code Execution Successful ===
```

dict Data Type:

- If we want to represent a group of values as key-value pairs then we should go for dict data type.
- Eg: d = {"Maths":99, "Hindi":89, "Python":98}
- Duplicate keys are not allowed but values can be duplicated. If we are trying to insert an entry with duplicate key, then old value will be replaced with new value.

main.py

```
d = {"Maths":99, "Hindi":89, "Python":98}
print(d)
print(type(d))
```

Output

```
{'Maths': 99, 'Hindi': 89, 'Python': 98}
<class 'dict'>

=== Code Execution Successful ===
```

None Data Type

None means nothing or No value associated.

The following are various important escape characters in Python

- ✓ \\-> Back slash
- ✓ \t-> tab
- ✓ \n-> new line

mutable & immutable

- **once we create an object, we cannot perform any changes in that object. If we are trying to change then with those changes a new object will be created. This non-changeable behaviour is called immutability.**

String Example:

```
# 1. Create an object
s1 = "Python"
print (f"Original: {s1}, ID: {id(s1)}")
```

main.py

```
# 1. Create an object
s1 = "Python"
print(f"Original: {s1}, ID: {id(s1)}")
```

Output

```
Original: Python, ID: 134024924719392

=== Code Execution Successful ===
```

```
# 2. "Modify" the object
s1 = s1 + " is cool"

# 3. Check the result
print (f"New: {s1}, ID: {id(s1)}")
```

main.py

```
# 1. Create an object
s1 = "Python"
print(f"Original: {s1}, ID: {id(s1)}")

# 2. "Modify" the object
s1 = s1 + " is cool"

# 3. Check the result
print(f"New: {s1}, ID: {id(s1)}")
```

Output

```
Original: Python, ID: 134677069969696
New: Python is cool, ID: 134677057427824

=== Code Execution Successful ===
```

Tuple Example:

main.py

```
my_tuple = (1, 2, 3)

# This will cause a TypeError
try:
    my_tuple[0] = 99
except TypeError as e:
    print(f"Error: {e}")
    # Output: 'tuple' object does not support item assignment
```

Output

```
ERROR!
Error: 'tuple' object does not support item assignment

=== Code Execution Successful ===
```

Data Type	Mutable / Immutable
int	Immutable
float	Immutable
complex	Immutable
bool	Immutable
str	Immutable
Bytes	Immutable
ByteArray	Mutable
list	Mutable
tuple	Immutable
set	Mutable
Frozen set	Immutable
dict	Mutable
range	Immutable

- **In Python if a new object is required, then PVM won't create object immediately. First it will check is any object available with the required content or not. If available, then existing object will be reused. If it is not available, then only a new object will be created. The advantage of this approach is memory utilization and performance will be improved.**

main.py

```python

a = 100
b = 100

print(f"ID of a: {id(a)}")  # Memory address of a
print(f"ID of b: {id(b)}")  # Memory address of b
print(a is b)               # Returns True: PVM reused the object


x = 500
y = 578

print(f"ID of x: {id(x)}")
print(f"ID of y: {id(y)}")
print(x is y)               # Returns False: PVM created two separate objects


```

Output

```
ID of a: 137248845942568
ID of b: 137248845942568
True
ID of x: 137248834449008
ID of y: 137248834449104
False

=== Code Execution Successful ===
```